

Лабораторная работа №5

Аппарат сетей Петри

Кудякова София Андреевна

2026-08-29

Содержание I

- 1 Введение
- 2 Реализация
- 3 Моделирование
- 4 Анимация
- 5 Сравнение и литературное программирование
- 6 Результаты

Раздел 1

Введение

Цель: изучить аппарат сетей Петри на примере задачи «Обедающие философы».

Задачи:

- построить классическую сеть Петри и исследовать deadlock;

Цель: изучить аппарат сетей Петри на примере задачи «Обедающие философы».

Задачи:

- построить классическую сеть Петри и исследовать deadlock;
- реализовать модифицированную сеть с арбитром;

Цель: изучить аппарат сетей Петри на примере задачи «Обедающие философы».

Задачи:

- построить классическую сеть Петри и исследовать deadlock;
- реализовать модифицированную сеть с арбитром;
- выполнить стохастическое (Гиллеспи) и детерминированное моделирование;

Цель: изучить аппарат сетей Петри на примере задачи «Обедающие философы».

Задачи:

- построить классическую сеть Петри и исследовать deadlock;
- реализовать модифицированную сеть с арбитром;
- выполнить стохастическое (Гиллеспи) и детерминированное моделирование;
- построить анимацию и провести сравнительный анализ;

Цель: изучить аппарат сетей Петри на примере задачи «Обедающие философы».

Задачи:

- построить классическую сеть Петри и исследовать deadlock;
- реализовать модифицированную сеть с арбитром;
- выполнить стохастическое (Гиллеспи) и детерминированное моделирование;
- построить анимацию и провести сравнительный анализ;
- реализовать литературное программирование и получить Julia/Notebook/Quarto.

Сеть Петри задаётся четвёркой:

$$N = (P, T, F, M_0)$$

- P — позиции (состояния/ресурсы);

Смена маркировки происходит при срабатывании разрешённого перехода.

Сеть Петри задаётся четвёркой:

$$N = (P, T, F, M_0)$$

- P — позиции (состояния/ресурсы);
- T — переходы (события);

Смена маркировки происходит при срабатывании разрешённого перехода.

Сеть Петри задаётся четвёркой:

$$N = (P, T, F, M_0)$$

- P — позиции (состояния/ресурсы);
- T — переходы (события);
- F — дуги между позициями и переходами;

Смена маркировки происходит при срабатывании разрешённого перехода.

Сеть Петри задаётся четвёркой:

$$N = (P, T, F, M_0)$$

- P — позиции (состояния/ресурсы);
- T — переходы (события);
- F — дуги между позициями и переходами;
- M_0 — начальная маркировка.

Смена маркировки происходит при срабатывании разрешённого перехода.

Задача «Обедающие философы»

N философов и N вилок за круглым столом. Состояния философа:

- Think — размышляет;

Каждая вилка — общий ресурс, недоступный одновременно двум философам.

Задача «Обедающие философы»

N философов и N вилок за круглым столом. Состояния философа:

- Think — размышляет;
- Hungry — голоден;

Каждая вилка — общий ресурс, недоступный одновременно двум философам.

Задача «Обедающие философы»

N философов и N вилок за круглым столом. Состояния философа:

- Think — размышляет;
- Hungry — голоден;
- Eat — ест (нужны обе соседние вилки).

Каждая вилка — общий ресурс, недоступный одновременно двум философам.

Если все философы одновременно захватят левую вилку, каждый будет ждать правую — **взаимная блокировка (deadlock)**.

Для предотвращения вводится позиция *Arbiter* с $N - 1$ фишками — не позволяет всем N философам одновременно начать захват вилок.

Раздел 2

Реализация

Структура проекта

Проект lab05 на DrWatson: `src`, `scripts`, `data`, `plots`, `notebooks`, `report`.
Основной модуль: `src/DiningPhilosophers.jl` со структурой `PetriNet`
(позиции, переходы, матрица инцидентности).

Классическая сеть и сеть с арбитром

Каждый философ: позиции Think_i , Hungry_i , Eat_i , Fork_i .

При $N = 5$: $4N = 20$ позиций, $3N = 15$ переходов.

- **Классическая сеть:** $\text{Think} \rightarrow \text{Hungry}$ (захват левой вилки) $\rightarrow$ попытка захватить правую $\rightarrow \text{Eat} \rightarrow$ возврат вилок.

Классическая сеть и сеть с арбитром

Каждый философ: позиции Think_i , Hungry_i , Eat_i , Fork_i .

При $N = 5$: $4N = 20$ позиций, $3N = 15$ переходов.

- **Классическая сеть:** $\text{Think} \rightarrow \text{Hungry}$ (захват левой вилки) $\rightarrow$ попытка захватить правую $\rightarrow \text{Eat} \rightarrow$ возврат вилок.
- **Сеть с арбитром:** добавлена позиция Arbiter с начальной маркировкой $N - 1 = 4$ — разрешение арбитра обязательно для захвата ресурса.

Раздел 3

Моделирование

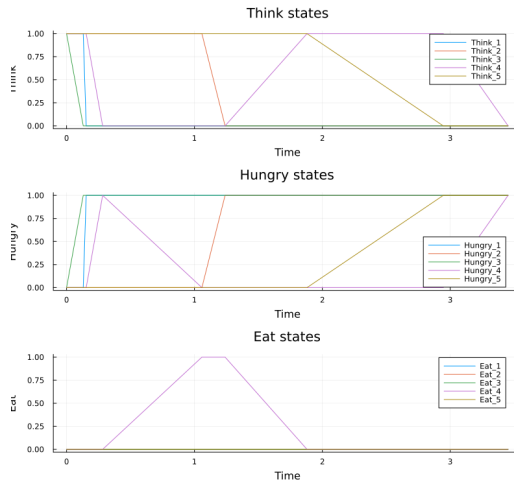
Стохастическое моделирование (алгоритм Гиллеспи)

Шаг алгоритма: расчёт интенсивностей $\rightarrow$ время до события $\rightarrow$ выбор перехода $\rightarrow$ изменение маркировки $\rightarrow$ сохранение состояния.

Параметры эксперимента: $N = 5$, $t_{\max} = 50$.

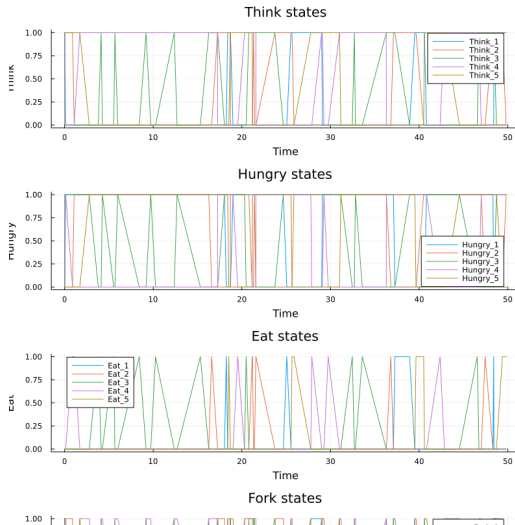
Классическая сеть — результат

Автоматическая проверка `detect_deadlock`: для классической сети ожидается переход в состояние без дальнейших изменений.



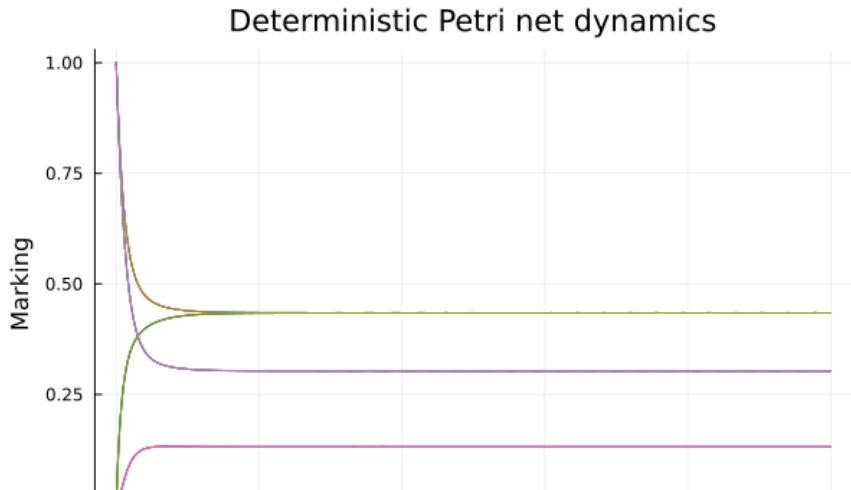
Сеть с арбитром — результат

Для сети с арбитром `detect_deadlock` должен возвращать `false`.



Детерминированное моделирование

Система ОДУ строится по матрице инцидентности сети (`simulate_ode`), решается методом `Tsit5`.

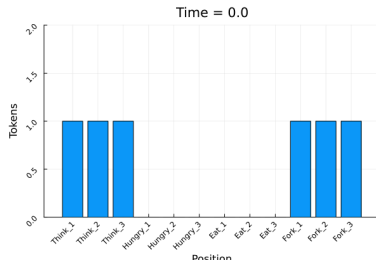
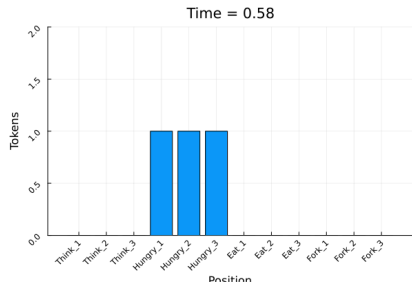


Раздел 4

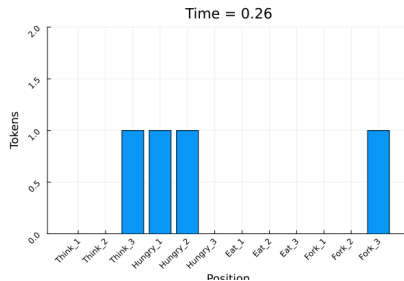
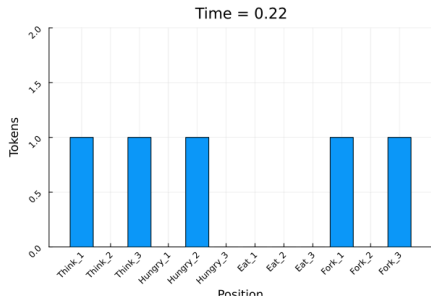
Анимация

Визуализация динамики

Уменьшенная модель: $N = 3$, $t_{\max} = 30$. Результат — GIF-анимация `philosophers_simulation.gif`, наблюдается перемещение фишек между Think, Hungry, Eat, Fork.



Кадры анимации (продолжение)

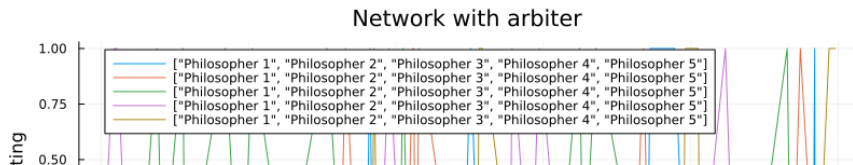
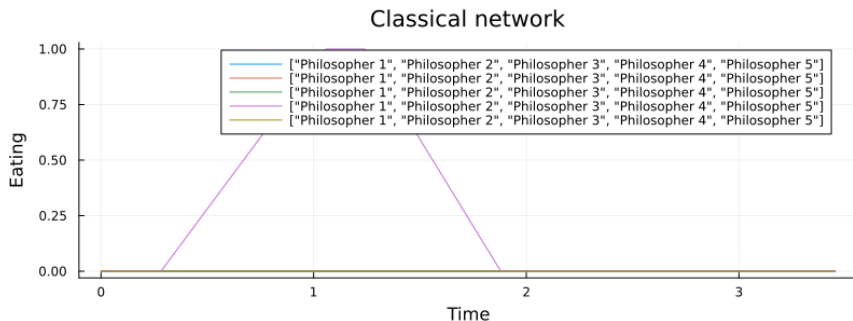


Раздел 5

Сравнение и литературное программирование

Сравнительный анализ

Скрипт `dining_philosophers_report.jl` сравнивает состояние `Eat_i` классической сети и сети с арбитром.



Файл `dining_philosophers_literate.jl` объединяет текст и код. Через `Literate.jl` получены: Julia-скрипт, Quarto-документ (QuartoFlavor), Jupyter Notebook.

```
((base) sofia@MacBook-Pro-Sonya project % julia --project=. scripts/dining_philosophers.jl
=====
КЛАССИЧЕСКАЯ СЕТЬ
=====
Deadlock обнаружен: true

=====
СЕТЬ С АРБИТРОМ
=====
Deadlock обнаружен: false

Моделирование завершено.
Результаты находятся в data/ и plots/.
(base) sofia@MacBook-Pro-Sonya project %
```

Рисунок 5: Литературный исходный файл модели

Jupyter Notebook

Сгенерированный ноутбук выполнен в Jupyter, проверены ячейки и результаты.



Раздел 6

Результаты

- классическая сеть демонстрирует взаимную блокировку;

- классическая сеть демонстрирует взаимную блокировку;
- арбитр с $N - 1$ разрешениями предотвращает deadlock;

- классическая сеть демонстрирует взаимную блокировку;
- арбитр с $N - 1$ разрешениями предотвращает deadlock;
- сравнение Eat_i подтверждает разницу в поведении моделей.

- реализована структура PetriNet, классическая и модифицированная модели;

- реализована структура PetriNet, классическая и модифицированная модели;
- выполнено стохастическое (Гиллеспи) и детерминированное (ОДУ) моделирование;

- реализована структура PetriNet, классическая и модифицированная модели;
- выполнено стохастическое (Гиллеспи) и детерминированное (ОДУ) моделирование;
- создана анимация и проведён сравнительный анализ;

- реализована структура PetriNet, классическая и модифицированная модели;
- выполнено стохастическое (Гиллеспи) и детерминированное (ОДУ) моделирование;
- создана анимация и проведён сравнительный анализ;
- реализовано литературное программирование: Julia-скрипт, Quarto, Jupyter Notebook.